
Lecture 2: It's not just 1s and 0s?

Analog I/O, interrupt handling, and data safety.

Robojackets Firmware Training 2026

Administrivia - Training Outline

- Lecture 1: C/C++ Basics
 - **Lecture 2: Interrupts & Analog I/O (today)**
 - Lecture 3: Communication Protocols
 - Lecture 4: Driver Development
 - Lecture 5: State Machines & RTOS
 - Practical Soldering Training
-

Registers

Everything is a Register!* - (Memory Mapped I/O)

*At least it used to. Registers are the smallest and quickest form of memory directly read/written to by the MCU. Though dependent on chip architecture, a lot of external devices are now MMIO, but registers are still used (at least in name)

- Operation value storage (i.e. $x = y + z \rightarrow \text{reg0} = \text{reg1} + \text{reg2}$)
 - Internal Peripheral interfacing
 - Timers
 - Internal Interrupts
 - External Peripheral interfacing
 - GPIO
 - Analog Inputs
 - Communication ports
 - External Interrupts
-

Proof of Concept

By modifying specific points in memory we can directly control pins!

Normal Blinky

```
1  #include <Arduino.h>
2
3
4  void setup() {
5      pinMode(13, OUTPUT);
6  }
7
8  void loop() {
9      digitalWrite(13, HIGH);
10     delay(500);
11     digitalWrite(13, LOW);
12     delay(500);
13 }
```

Register Level* Blinky

```
1  #include <Arduino.h>
2
3  volatile uint16_t* portb = (uint16_t*) 0x0025;
4  volatile uint16_t* ddrb = (uint16_t*) 0x0024;
5
6  void setup() {
7      *ddrb |= (1 << 5);
8  }
9
10 void loop() {
11     *portb |= (1 << 5);
12     delay(500);
13     *portb &= ~(1 << 5);
14     delay(500);
15 }
```

* delay() can be broken into a Timer register read however it has complex logic supporting it.

How Arduino does it

```
void pinMode(uint8_t pin, uint8_t mode)
{
    uint8_t bit = digitalPinToBitMask(pin);
    uint8_t port = digitalPinToPort(pin);
    volatile uint8_t *reg, *out;

    if (port == NOT_A_PIN) return;

    // JWS: can I let the optimizer do this?
    reg = portModeRegister(port);
    out = portOutputRegister(port);

    if (mode == INPUT) {
        uint8_t oldSREG = SREG;
        cli();
        *reg &= ~bit;
        *out &= ~bit;
        SREG = oldSREG;
    } else if (mode == INPUT_PULLUP) {
        uint8_t oldSREG = SREG;
        cli();
        *reg &= ~bit;
        *out |= bit;
        SREG = oldSREG;
    } else {
        uint8_t oldSREG = SREG;
        cli();
        *reg |= bit;
        SREG = oldSREG;
    }
}
```

```
void digitalWrite(uint8_t pin, uint8_t val)
{
    uint8_t timer = digitalPinToTimer(pin);
    uint8_t bit = digitalPinToBitMask(pin);
    uint8_t port = digitalPinToPort(pin);
    volatile uint8_t *out;

    if (port == NOT_A_PIN) return;

    // If the pin that support PWM output, we need to turn it off
    // before doing a digital write.
    if (timer != NOT_ON_TIMER) turnOffPWM(timer);

    out = portOutputRegister(port);

    uint8_t oldSREG = SREG;
    cli();

    if (val == LOW) {
        *out &= ~bit;
    } else {
        *out |= bit;
    }

    SREG = oldSREG;
}
```

Registers from Technical Reference Manuals (TRMs)

Most datasheets for any chip will feature some form of register mapping which will tell you what each register is, what its values represent, and its address.

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

1.1.3 Port B (PB7:0) XTAL1/XTAL2/TOSC1/TOSC2

Port B is an 8-bit bi-directional I/O port with internal pull-up resistors (selected for each bit). The Port B output buffers have symmetrical drive characteristics with both high sink and source capability. As inputs, port B pins that are externally pulled low will source current if the pull-up resistors are activated. The Port B pins are tri-stated when a reset condition becomes active, even if the clock is not running.

Depending on the clock selection fuse settings, PB6 can be used as input to the inverting oscillator amplifier and input to the internal clock operating circuit.

Depending on the clock selection fuse settings, PB7 can be used as output from the inverting oscillator amplifier.

If the internal calibrated RC oscillator is used as chip clock source, PB7..6 is used as TOSC2..1 input for the asynchronous Timer/Counter2 if the AS2 bit in ASSR is set.

The various special features of port B are elaborated in [Section 13.3.1 “Alternate Functions of Port B” on page 65](#) and [Section 8. “System Clock and Clock Options” on page 24](#).

Interrupts

What are interrupts?

Interrupts interrupt the control flow of your program. When an interrupt is triggered a corresponding interrupt handler is called immediately* and executed.

11.1 Interrupt Vectors in ATmega328P

Table 11-1. Reset and Interrupt Vectors in ATmega328P

Vector No.	Program Address	Source	Interrupt Definition
1	0x0000	RESET	External pin, power-on reset, brown-out reset and watchdog system reset
2	0x0002	INT0	External interrupt request 0
3	0x0004	INT1	External interrupt request 1
4	0x0006	PCINT0	Pin change interrupt request 0
5	0x0008	PCINT1	Pin change interrupt request 1
6	0x000A	PCINT2	Pin change interrupt request 2
7	0x000C	WDT	Watchdog time-out interrupt
8	0x000E	TIMER2 COMPA	Timer/Counter2 compare match A
9	0x0010	TIMER2 COMPB	Timer/Counter2 compare match B
10	0x0012	TIMER2 OVF	Timer/Counter2 overflow
11	0x0014	TIMER1 CAPT	Timer/Counter1 capture event
12	0x0016	TIMER1 COMPA	Timer/Counter1 compare match A
13	0x0018	TIMER1 COMPB	Timer/Counter1 compare match B
14	0x001A	TIMER1 OVF	Timer/Counter1 overflow
15	0x001C	TIMER0 COMPA	Timer/Counter0 compare match A
16	0x001E	TIMER0 COMPB	Timer/Counter0 compare match B
17	0x0020	TIMER0 OVF	Timer/Counter0 overflow
18	0x0022	SPI, STC	SPI serial transfer complete
19	0x0024	USART, RX	USART Rx complete
20	0x0026	USART, UDRE	USART, data register empty
21	0x0028	USART, TX	USART, Tx complete
22	0x002A	ADC	ADC conversion complete
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Analog comparator
25	0x0030	TWI	2-wire serial interface
26	0x0032	SPM READY	Store program memory ready

Common Uses

Internal Interrupt:

- Timers
- Watchdog

External Interrupt:

- GPIO
- Communication Triggers

* Interrupts can be disabled for many reasons such as atomic sections or interrupts.

Precise Timing

Timer interrupts can be used to have events trigger at precise times regardless of main loop actions.

```
1  #include <Arduino.h>
2
3  volatile uint16_t msCount = 0;
4
5  void setup() {
6      // Pin 13 = PB5 on the ATmega328P
7      DDRB |= (1 << DOB5);
8
9      // Stop Timer1 while configuring it
10     TCCR1A = 0;
11     TCCR1B = 0;
12     TCNT1 = 0;
13
14     // CTC mode: reset timer when it reaches OCR1A
15     TCCR1B |= (1 << WGM12);
16
17     // 16 MHz / 64 = 250 kHz
18     // 250 counts = 1 ms
19     OCR1A = 249;
20
21     // Enable Timer1 compare-match A interrupt
22     TIMSK1 |= (1 << OCIE1A);
23
24     // Start Timer1 with prescaler = 64
25     TCCR1B |= (1 << CS11) | (1 << CS10);
26 }
27
28 void loop() {
29     // Nothing needed here.
30 }
31
32 // Called exactly once every 1 ms
33 ISR(TIMER1_COMPA_vect) {
34     msCount++;
35     if (msCount >= 500) {
36         msCount = 0;
37
38         // Toggle pin 13
39         PORTB ^= (1 << PORTB5);
40     }
41 }
```

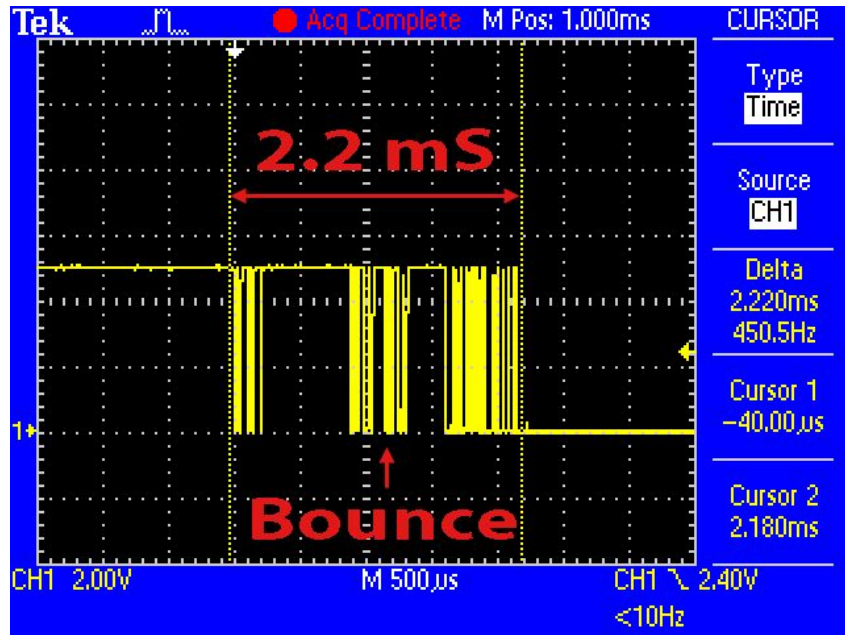
Using libraries* to handling timer interrupts is safer and easier than manually manipulating registers!

```
1  #include <Arduino.h>
2  #include <TimerOne.h>
3
4  const int LED_PIN = 13;
5
6  void blinkLED() {
7      digitalWrite(LED_PIN, !digitalRead(LED_PIN));
8  }
9
10 void setup() {
11     pinMode(LED_PIN, OUTPUT);
12
13     // Timer1 interrupt every 500,000 microseconds = 500 ms
14     Timer1.initialize(500000);
15     Timer1.attachInterrupt(blinkLED);
16 }
17
18 void loop() {
19     // Delay is non blocking for interrupts!
20     delay(100000);
21 }
```

*Note that there are few universal timer interrupt libraries, most are purpose built per microcontroller.

Debouncing

One of the most common uses of firmware interrupts is detecting button presses. However, like most interactions with physical devices, there are imperfections.



```
#include <Arduino.h>

void button_isr();
// You can add compiler side replacements with #define
#define BUTTON_PIN 3
// Save the time of the last trigger
uint32_t last_trigger = millis();

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PIN, INPUT);
  // Attach an interrupt handler to the rising edge of the button press
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), button_isr, RISING);
}

void loop() {
  delay(1); // Nothing!
}

void button_isr() {
  // Ignore presses at sub 20ms intervals
  if (millis() - last_trigger < 20) {
    return;
  }
  last_trigger = millis();
  Serial.println("Pressed!"); // Prints in ISRs are bad...
}
```

Interrupt Memory Safety

Atomic vs Non-Atomic Actions

Atomic actions are non-interruptible. They are the smallest action that can be completed in the shortest time step. In general most actions are non-atomic. However certain ones, such as single byte read/writes often are. This is entirely dependent on the architecture of the particular microcontroller.

```
uint8_t foo = 23;  
uint8_t bar;  
// Most 8-bit operations are atomic  
bar = foo;
```

```
uint16_t foo = 23;  
uint16_t bar;  
// 2 byte access, non-atomic!  
bar = foo;
```

For example: Should a ISR be called perfectly during the uint16_t read, 'bar' could receive incorrect data.

$\text{bar}[7:0] = \text{foo}[7:0] \rightarrow \text{ISR modifies foo} \rightarrow \text{bar}[15:8] = (\text{new})\text{foo}[15:8]$

Volatile

When a piece of memory is accessed outside of normal operations, either by a second processor or ISR, it is important to mark it “volatile.”

```
// The volatile keyword prevents compiler optimizations
volatile bool pressed = false;

void loop() {
  if (pressed) {
    // Do something
  }
}

void button_isr() {
  pressed = digitalRead(BUTTON_PIN);
}
```

Otherwise the compiler may mark it unchanging and replace it with a fixed value. The volatile keyword forces the compiler to access memory each time.

Forced Atomicity

While atomic processes are too short to be interrupted, you can prevent interrupts to make any process atomic.

However, long “noInterrupt” sections can mess with the timing of interrupts, so keep them short.

```
volatile uint32_t count = 0;

void loop() {
  // Disable interrupts to quickly create a copy
  // Just like interrupts, noInterrupt blocks should be quick
  noInterrupts();
  uint32_t count_copy = count;
  interrupts();

  // Now do any expensive processes using the copy
  Serial.print(count_copy);
}

void button_isr() {
  count++;
}
```

Sharing Resources with Mutex

Mutexes can be used to prevent multiple processors/threads from accessing the same resource at once. This can be anything from a variable to a real peripheral.

While the Arduino Nano does not support this, more advanced processors such as the Raspberry Pi Pico do.

Be warned: A “deadlock” can occur when one processor is waiting on a mutex to free the mutex needed by another processor. This will stall infinitely!

```
// Declare the mutex instance globally
auto_init_mutex(my_mutex);

// Shared resource
volatile int shared_counter = 0;

// This function runs exclusively on Core 1
void core1_entry() {
    while (1) {
        // Safe zone: Lock the mutex before touching shared data
        mutex_enter_blocking(&my_mutex);

        shared_counter++;

        // Release the mutex so Core 0 can use it
        mutex_exit(&my_mutex);
        sleep_ms(10);
    }
}

int main() {
    stdio_init_all();

    // Launch core1_entry() on Core 1
    multicore_launch_core1(core1_entry);

    // This loop runs exclusively on Core 0
    while (1) {
        // Lock the mutex before reading/writing shared data
        mutex_enter_blocking(&my_mutex);

        printf("Current Counter Value: %d\n", shared_counter);

        // Release the mutex
        mutex_exit(&my_mutex);
        sleep_ms(500);
    }
}
```

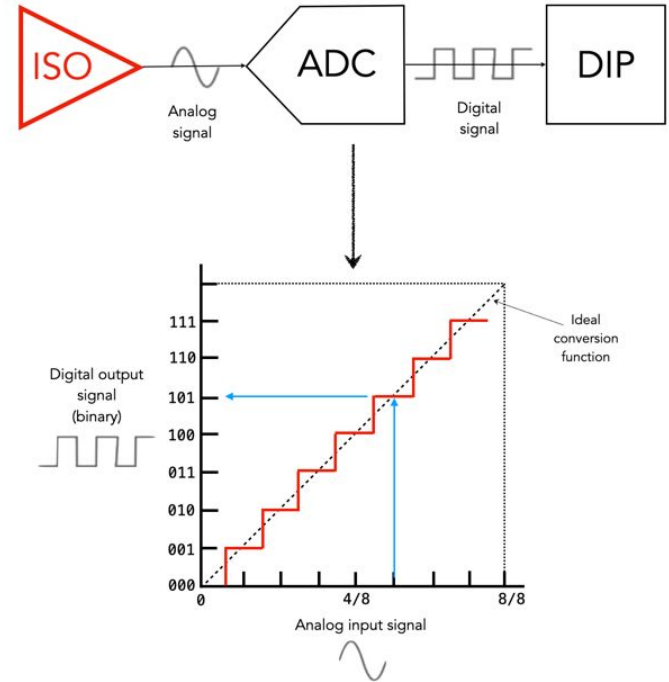
Special Pin Functionality

Analog Input

Digital devices cannot interpret analog signals without first converting them into discrete levels. This is what an Analog to Digital Converter (ADC) does.

The Arduino Nano features 6 10 bit ADC on pins A0-5. These pins convert voltage levels 0-5V to the range of 0-1023 (2^{10}).

```
1  #include <Arduino.h>
2
3  void setup() {
4    // Analog pins do not need to be initialized manually
5  }
6
7  void loop() {
8    // raw is 0 - 1023
9    uint16_t raw = analogRead(A0);
10   // voltage is 0.0 - 5.0
11   float voltage = (raw / 1023.0f) * 5.0f;
12 }
```



Analog Output

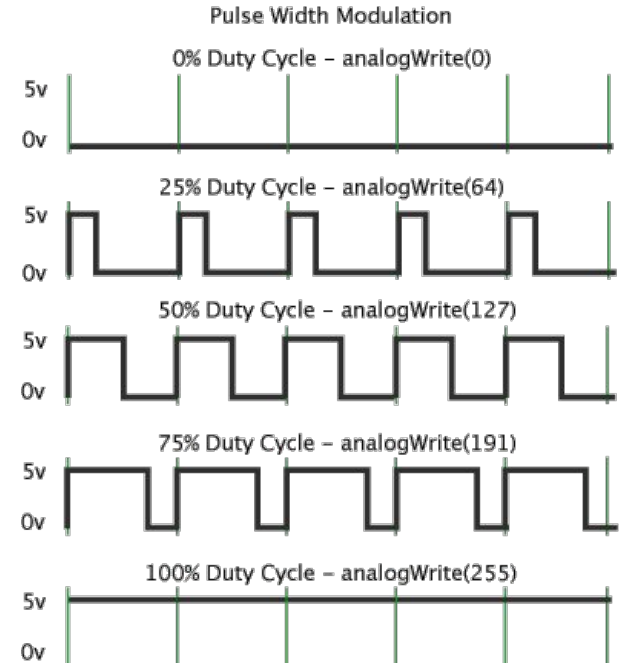
Similar to reading analog signals, digital cannot perfectly recreate analog signals. One method of achieving something similar is Pulse Width Modulation (PWM).

Instead of outputting 3V, output 5V 60% of the time to achieve the same average voltage!

The Arduino Nano features 6 8 bit PWM capable pins (3, 5, 6, 9, 10, 11).

These pins can have duty cycles of 0-255 (mapped to %).

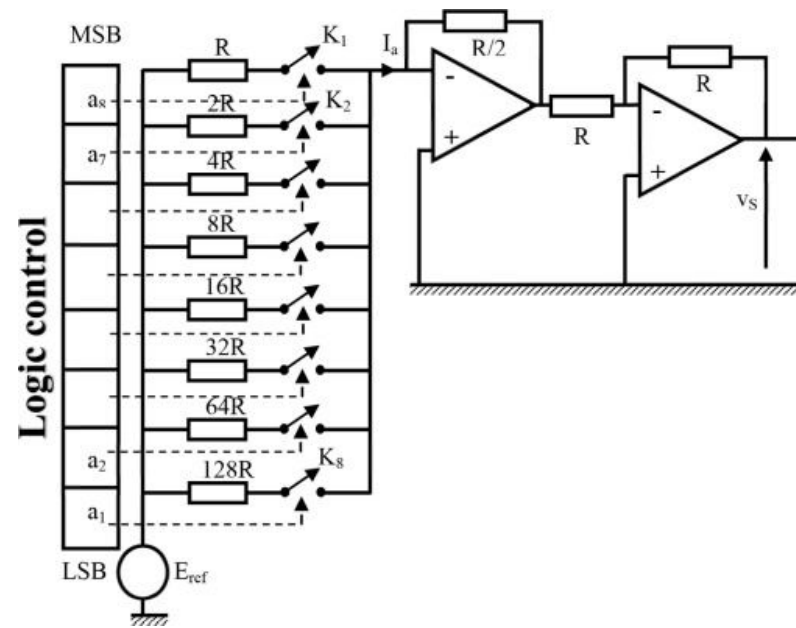
```
1  #include <Arduino.h>
2
3  void setup() {
4      // Analog pins do not need to be initialized manually
5  }
6
7  void loop() {
8      // 5V 25% of the time ~1.25V
9      analogWrite(3, 64);
10 }
```



Better Analog Output

While PWM can mimic voltage level changes, it is not really changing the voltage. This is where a Digital to Analog Converter (DAC) comes in. It follows the same logic as a ADC just reversed!

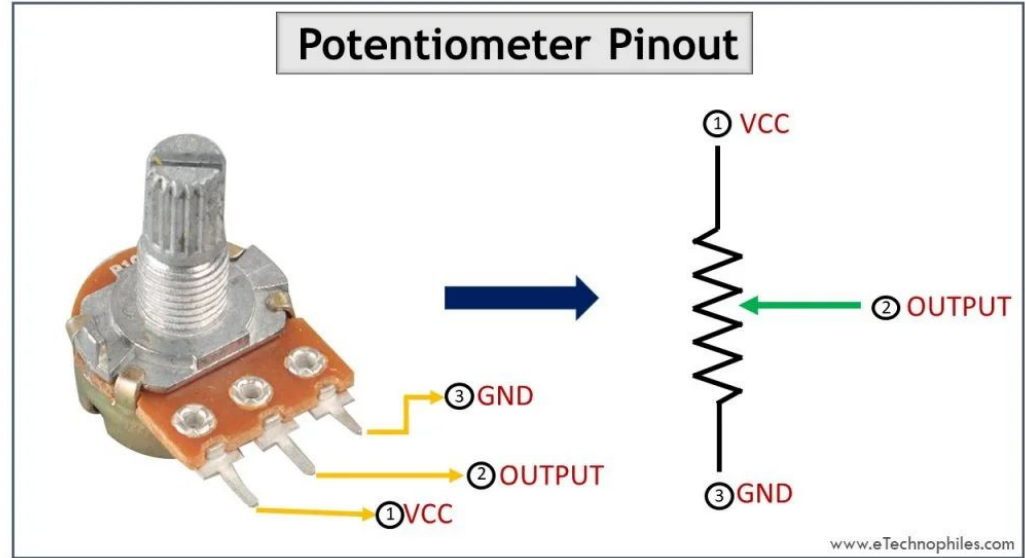
Note: While ADCs are common in microcontrollers, DACs are rarely included. Make sure you check your data sheets!



LAB TIME!!!

Today's Labs

- **Switch Buttons**
 - Use interrupts to read button inputs
- **PWM**
 - Control LED brightness using PWM
- **Potentiometer Control***
 - Control LED brightness based off of potentiometer input



* Remove the IMU if present and tie output of potentiometer to pin A5 in the slot, power and ground as expected.